Name: Heang Socheata

ID: e20240400

I3-AMS-A

TP02

1. Python Basic Data Types

In Python, values are objects. Built-in data types describe the kind of information a program stores and the operations that can be performed on that information.

a. What is Primitive/Built-in Data Types?

A primitive data type is a basic type of data used to store a single value, such as a number, a decimal number, a true/false value, or text .

Type	Meaning	Example
int	Whole numbers without a decimal part	20, -7, 0
float	Decimal or floating-point values	95.5, 3.14
bool	Logical truth values	True, False
str	Text made from characters	"Dara", 'Python'

b. Collection and Reference Types (list, tuple, dict, set, objects)

Type	Ordered ?	Mutable ?	Main idea	Example
list	Yes	Yes	Changeable sequence; duplicates allowed	[70, 80, 90]
tuple	Yes	No	Fixed sequence	(70, 80, 90)
dict	Yes	Yes	Key-value records	{'name': 'Dara'}
set	No	Yes	Unique elements	{'Python', 'Math'}

object	—	Depends	Instance created from a class	Student('Dara')

c. Literals in Python

Numeric literals: 20, 3.14, 0b1010, 2+3j.

String literals: 'Dara', "Python", and triple-quoted multi-line strings.

Boolean literals: True and False.

None literal: represents the absence of a value.

Collection literals: [ ], ( ), {key: value}, and {unique values}.

Practical example: storing student names, ages, scores, and student records.

Input:

basic.py > ...

```python
name = "Cheata"
age = 19
scores = [80.2, 90.0, 85.0]
is_passed = True
remarks = None
subjects = ("Python", "Math", "English")
departments = {"Data Science", "IT"}

student = {
    "name": name,
    "age": age,
    "scores": scores,
    "is_passed": is_passed,
    "remarks": remarks
}

print("Student Name:", student["name"])
print("Age:", student["age"])
print("Scores:", student["scores"])
print("Passed:", student["is_passed"])
```

Output:

```
Student Name: Cheata
Age: 19
Scores: [80.2, 90.0, 85.0]
Passed: True
```

2. Python Variables and Attributes

a. Local Variables

A local variable is created inside a function or method. It can be used only while that function or method is running.

Input:

```python
def add(a, b):
    total = a + b
    return total

print(add(2, 3))
```

output:

```
PS C:\Users\MSI
5
```

b. Instance Attributes

An instance attribute belongs to one specific object. It is normally created with self-inside __init__, so different objects can keep different values.

Input:

```python
class Student:
    def __init__(self, name):
        self.name = name

s1 = Student("Dara")
s2 = Student("Sokha")
print(s1.name)
print(s2.name)
```

output:

```
Dara
Sokha
```

c. Class Attributes (shared by all instances)

A class attribute is defined directly in the class body, outside methods. It belongs to the class and is shared by its instances.

Input:

```python
class Student:
    school = "ITC"          # Class attribute

    def __init__(self, name):
        self.name = name     # Instance attribute

    def hello(self):
        message = "Hello"    # Local variable
        print(message, self.name, self.school)

s1 = Student("Dara")
s2 = Student("Sokha")
s1.hello()
s2.hello()
```

Output:

```
Hello Dara ITC
Hello Sokha ITC
```

3. Python Modifiers and Access Control — Part 1

a. Public, Non-Public, and Private Naming Conventions

Style	Example	Meaning
Public	owner	Use freely from inside or outside the class.
Non-public	_balance	Internal use by convention; direct access still works.
Private-style	__pin	Python applies name mangling to reduce accidental access.

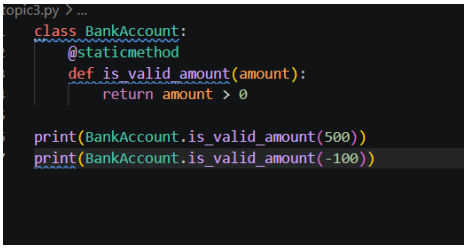
b. Access Control by Convention and Name Mangling

A single underscore is a convention rather than a strict restriction. A double underscore triggers name mangling. For example, __pin in Account is stored under a name such as _Account__pin. This helps prevent accidental access; it is not a security mechanism.

c. The @staticmethod Decorator

@staticmethod defines a function inside a class that does not receive self or cls.

Input:



Output:



d. The @classmethod Decorator

@classmethod receives the class itself as the first argument, conventionally named cls. It is useful for shared class-level data or alternative constructors.

Practical Example: BankAccount

Input:

```python
class BankAccount:
    bank_name = "ABC Bank"
    total_accounts = 0

    def __init__(self, owner, balance):
        self.owner = owner
        self._account_type = "Savings"
        self.__balance = balance
        BankAccount.total_accounts += 1

    def deposit(self, amount):
        if self.is_valid_amount(amount):
            self.__balance += amount

    def get_balance(self):
        return self.__balance

    @staticmethod
    def is_valid_amount(amount):
        return amount > 0

    @classmethod
    def show_bank_info(cls):
        print("Bank:", cls.bank_name)
        print("Total accounts:", cls.total_accounts)

account = BankAccount("Dara", 1000)
account.deposit(500)
print("Owner:", account.owner)
print("Balance:", account.get_balance())
BankAccount.show_bank_info()
```

Output:

```
True
False
Owner: Dara
Balance: 1500
Bank: ABC Bank
Total accounts: 1
```

4. Python Modifiers and OOP Features — Part 2

e. Abstract Classes and Methods (abc, @abstractmethod)

An abstract class is a blueprint. It specifies operations that subclasses must implement. Python provides ABC and @abstractmethod for this purpose.

f. Access Control and Encapsulation

Encapsulation keeps data and related methods together inside a class and controls how the data is accessed. Python commonly uses public names, single-underscore non-public names, double-underscore name mangling, and properties.

g. Other OOP Features: @property, @final, and typing.final

@property lets a method be accessed like an attribute and can provide validation or computed values.

@final from typing marks a method or class as not intended to be overridden or inherited. Type checkers such as mypy or Pyright can report violations; Python does not strictly enforce it at runtime.

Practical Example: Shape, Circle, Rectangle

Input:

```python
from abc import ABC, abstractmethod
from typing import final
import math

class Shape(ABC):
    @property
    @abstractmethod
    def area(self):
        ...

    @abstractmethod
    def perimeter(self):
        ...

    @final
    def describe(self):
        return f"area={self.area:.2f}, perimeter={self.perimeter():.2f}"

class Circle(Shape):
    def __init__(self, radius):
        self.__radius = radius

    @property
    def area(self):
        return math.pi * self.__radius ** 2

    def perimeter(self):
        return 2 * math.pi * self.__radius

class Rectangle(Shape):
    def __init__(self, width, height):
        self.__width = width
        self.__height = height

    @property
    def area(self):
```

```
area=78.54, perimeter=31.42
area=24.00, perimeter=20.00
```

5. Python Operators — Part 1

a. Arithmetic Operators

Operator	Meaning	Example with a=10, b=3
+	Addition	13
-	Subtraction	7
*	Multiplication	30
/	Division	3.333...
//	Floor division	3
%	Remainder	1
**	Power	1000

b. Comparison Operators

Comparison operators compare values and return True or False: ==, !=, >, <, >=, <=.

c. Bitwise Operators

Bitwise operators work on integer binary bits: & AND, | OR, ^ XOR, ~ NOT, << left shift, and >> right shift.

Input:

```
a = 5   # 0101
b = 3   # 0011
print(a & b)
print(a | b)
print(a ^ b)
print(~a)
print(a << 1)
print(a >> 1)
```

Output:

```
1
7
6
-6
10
2
```

d. Logical Operators

• Logical operators are used to combine or reverse conditions and return Boolean values (True or False).

Practical Example: Grade and Eligibility

Input:

```
math = 78
physics = 35
english = 42
attendance = 90

total = math + physics + english
average = total / 3
passed = average >= 50
eligible = average >= 80 and attendance >= 75

print("Total:", total)
print("Average:", average)
print("Exam result:", "PASS" if passed else "FAIL")
print("Award eligibility:", "ELIGIBLE" if eligible else "NOT ELIGIBLE")
```

Output:

```
Total: 155
Average: 51.666666666666664
Exam result: PASS
Award eligibility: NOT ELIGIBLE
```

6. Python Operators — Part 2

e. Assignment Operators

Operator	Example	Equivalent idea
=	x = 10	Assign 10 to x
+=	x += 5	x = x + 5

-=	x -= 2	x = x - 2
*=	x *= 3	x = x * 3
/=	x /= 2	x = x / 2
//=	x //= 3	x = x // 3
%=	x %= 4	x = x % 4
**=	x **= 2	x = x ** 2

f. Special Operators

Identity: is and is not check whether references point to the same object.

Membership: in and not in check whether a value exists in a collection.

Conditional expression: value_if_true if condition else value_if_false.

g. Operator Precedence and Associativity

Precedence decides which operator is evaluated first. Associativity decides the direction when operators have the same precedence. Parentheses can be used to make the intended order explicit.

Practical Example:

Input:

```
class Student:
    def __init__(self, score):
        self.score = score

s = Student(60)
s.score += 10

subjects = ["Python", "Math", "Physics"]
result = "Pass" if s.score >= 50 else "Fail"

print("Updated score:", s.score)
print("Study Python:", "Python" in subjects)
print("Result:", result)
print("Expression:", 2 + 3 * 4)
```

Output:

```
Updated score: 70
Study Python: True
Result: Pass
Expression: 14
```

7. Python Decision-Making Statements

a. The if Statement

The if statement runs a block only when its condition is True.

Input:

```
score = 75
if score >= 60:
    print("You passed!")
```

Output:

```
You passed!
```

b. The if-else Statement

if-else chooses between exactly two paths.

Input:

```
score = 45
if score >= 60:
    print("Pass")
else:
    print("Fail")
```

Output:

```
Fail
```

c. The if-elif-else Statement

if-elif-else checks multiple conditions from top to bottom and runs the first matching branch.

Practical Example: Pass/Fail and Letter Grade

Input:

```
score = 83

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"

status = "PASS" if score >= 60 else "FAIL"
print("Status:", status)
print("Grade:", grade)
```

Output:

```
Status: PASS
Grade: B
```

8. Python Decision-Making — Part 2

d. Nested if Statements

A nested if is an if statement inside another if. The inner condition is checked only when the outer condition is True.

Output:

```
age = 20
score = 72

if age >= 18:
    if score >= 50:
        print("Eligible")
    else:
        print("Low score")
else:
    print("Too young")
```

Input:

```
Eligible
```

e. Multiple-Branch Selection Using match-case

match-case (Python 3.10+) compares one value with several patterns. The first matching case runs, and case _ acts as the default.'

Input:

```
day = 3

match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case _:
        print("Other")
```

Output:

```
Wednesday
```

f. Conditional Expressions: Java vs Python

Language	Form
Java	condition ? value1 : value2
Python	value1 if condition else value2

Practical Example: Menu-Driven Student Management

Input:

```
students = []

while True:
    print("1. Add student")
    print("2. View students")
    print("3. Exit")
    choice = int(input("Choose: "))

    match choice:
        case 1:
            name = input("Name: ")
            if name != "":
                students.append(name)
        case 2:
            print(students)
        case 3:
            print("Exit program")
            break
        case _:
            print("Invalid")
```

Output:

```
1. Add student
2. View students
3. Exit
Choose: 1
Name: Cheata
1. Add student
2. View students
3. Exit
Choose: 3
Exit program
```

9. Python Loops and Loop Control

A loop repeats a block of code. Python mainly uses while for condition-based repetition and for for iterating through an iterable.

a. The while Loop

A while loop repeats a block of code as long as its condition is True. It is especially useful when we do not know in advance how many repetitions will be required.

Input:

```
count = 1
while count <= 3:
    print(count)
    count += 1
```

Output:

```
1
2
3
```

b. The for Loop

A for loop repeats a block of code by taking items one at a time from an iterable, such as a list, tuple, string, dictionary, or range().

Input:

```
students = ["Dara", "Sokha", "Malis"]
for name in students:
    print(name)
```

Output:

```
Dara
Sokha
Malis
```

c. Repetition with for and while (Python has no native do-while loop)

• Both loops repeat instructions, but they are best suited to different situations.

• Python does not have a native do-while loop. In some programming languages, a do while loop executes the body first and checks the condition afterward. Therefore, its body runs at least once.

• In Python, we can achieve similar behavior by combining while True with break.

```
while True:
    answer = input("Enter yes to continue: ")
    if answer == "yes":
        break
    print("Please try again.")

print("Finished!")
```

Explanation:

1. while True starts a loop that would continue indefinitely.

2. The program requests input from the user.

3. If the answer is "yes", break terminates the loop.

4. Otherwise, the program displays a message and asks again.

d. Loop Control: break, continue, pass

- **Break:** exits the nearest loop immediately.
- **Continue:** skips the rest of the current iteration and starts the next iteration.
- **Pass:** does nothing; it is a placeholder where Python requires a statement.

Practical Example: Student Objects, Averages, and Search

Input:

```
class Student:
    def __init__(self, name, grades):
        self.name = name
        self.grades = grades

    def average(self):
        return sum(self.grades) / len(self.grades)

students = [
    Student("Dara", [80, 90, 85]),
    Student("Sokha", [70, 60, 80]),
    Student("Lina", [90, 95, 88])
]

for student in students:
    print(student.name, "Average:", student.average())

search_name = "Dara"
for student in students:
    if student.name == search_name:
        print("Found:", student.name, "with average", student.average())
        break
else:
    print("Student not found")
```

Output:

```
Dara Average: 85.0
Lina Average: 91.0
Enter student name to search: Dara
Found: Dara with average score of 85.0
Dara
Sokha
Lina
```

Important Differences Between Java and Python

Topic	Java	Python
Blocks	Uses braces { }	Uses indentation
Variable declarations	Usually explicit types	Dynamic typing; no type declaration required
Counting loop	for (int i = 0; i < n; i++)	for i in range(n)
For each	for (Student s : list)	for s in students
do-while	Built in	No native do-while; use while True + break
Multiple branch	switch (modern Java also has switch expressions)	match-case from Python 3.10+
Conditional expression	condition ? a : b	a if condition else b
Access control	public/private/protected keywords	Mostly naming conventions and name mangling
Getters/setters	Commonly explicit methods	Often @property
Loop else	Not available	for/while can have else